

SISTEMES DIGITALS (Curs 2022-2023)

PRÀCTICA 3: Disseny estructural i reaprofitament del codi

El disseny estructural consisteix en dividir la descripció en entitats més petites i relacionar-les entre elles mitjançant una estructura jeràrquica. Utilitzant aquest mètode, treballarem amb entitats més simples les quals seran més fàcils de descriure, de simular i de corregir en el cas que s'hi detectin errors. Alhora, aquestes entitats, es poden dissenyar de forma que puguin ser reutilitzades en futurs projectes, dotant-les de paràmetres genèrics que ens permetran adaptar-les a diferents requeriments de disseny.

Aprofitant el disseny de la pràctica anterior, dividirem el codi en tres parts de les quals en farem tres projectes independents:

1. **Divisor de rellotge:** El divisor utilitzat per dotar al refresc dels 4 displays de 7 segments d'una freqüència adequada, l'aïllarem en una entitat independent en la qual hi afegirem un paràmetre genèric corresponent al final de comptatge, o millor dit, al valor pel qual dividim la freqüència del rellotge. Per a fer això, en la declaració de l'entitat, a més a més d'especificar els ports d'E/S, també hi afegirem un valor enter anomenat, per exemple, *eoc* (end of counting) que introduïrem utilitzant la paraula clau **generic**, i al qual li assignarem un valor per defecte. Aquest paràmetre el podrem modificar cada cop que instanciem aquesta entitat com a component en un disseny jeràrquic, i així podrem adaptar-la a nous requeriments de disseny.

La declaració de l'entitat quedaria de la següent manera:

```
entity clk_divider is
    generic (eoc: integer:= 100000);
    port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          clk_div : out STD_LOGIC);
end clk_divider;
```

2. **Convertidor de binari a BCD:** Per tal de crear un convertidor prou versàtil per poder ser utilitzat en altres dissenys, li assignarem l'amplada en bits del valor a convertir com a paràmetre genèric. Per exemple, farem que pugui convertir valors binaris de fins a 16 bits, i que tingui 5 sortides de 4 bits per als 5 dígits BCD que necessitem per a representar el valor màxim convertit (65535). Quan

instanciem aquesta entitat en el nostre disseny, introduïrem el valor 13 al paràmetre genèric, i deixarem “open” el port corresponent al 5è dígit BCD.

Així, la declaració de l’entitat del convertidor seria la següent:

```
entity bin2bcd is
    generic (n: positive :=16);
    Port ( bin_in : in STD_LOGIC_VECTOR (n-1 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          bcd0, bcd1, bcd2, bcd3, bcd4 : out STD_LOGIC_VECTOR (3 downto 0));
end bin2bcd;
```

Com podeu observar, el valor del paràmetre *n* afecta a l’amplada de bus del port *bin_in*, corresponent al valor a convertir. A més, dins de l’arquitectura també s’haurà de tenir en compte alhora de declarar i utilitzar els senyals corresponents als registres de desplaçament.

- 3. Codificador a 7 segments:** Finalment, crearem l’entitat corresponent a la codificació dels valors BCD a 7 segments i al refresc dels displays. Aquesta entitat no tindrà paràmetres genèrics, ja que sempre la utilitzarem per fer descripcions aplicades a la Basys 3 (i aquesta disposa dels displays que disposa, i prou). A part dels ports corresponents als BCD’s d’entrada, el clk i el reset, també tindrà un port corresponent al clock dividit (sortida del component dedicat a tal efecte). Com a sortida tindrà els ports dels ànodes i càtodes dels displays.

Exemple de declaració d’entitat:

```
entity seg7_coder is
    Port ( char0, char1, char2, char3 : in STD_LOGIC_VECTOR (3 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          clk_div : in STD_LOGIC;
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
end seg7_coder;
```

Cal recordar que un cop creat cadascun dels projectes, aquests s’han de sintetitzar per verificar que no hi hagi errors i simular per a comprovar el seu correcte funcionament.

Un cop fet això, haurem de crear un projecte que inclogui com a source files els arxius .vhd de les 3 entitats anteriors, més un altre .vhd que se situarà en la posició més alta de la jerarquia, i des d’on s’instanciaran aquestes entitats com a components i es crearan les connexions corresponents entre elles i amb els ports del disseny final.

Aquesta entitat **top level** tindrà la mateixa declaració d'entitat que en la pràctica anterior, ja que, visualitzant el disseny com a caixa negra és exactament el mateix.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity prac3_1 is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          sw : in STD_LOGIC_VECTOR (12 downto 0);
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
end prac3_1;
```

A la part declarativa de l'arquitectura, haurem de declarar els tres components a utilitzar, més un parell de senyals que ens serviran de connexions internes entre components: el clk_div i els BCD's que connectaran la sortida del convertidor binari a BCD amb el codificador a 7 segments.

```
architecture Behavioral of prac3_1 is

    component clk_divider
        generic (eoc: integer:= 100000);
        port ( clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              clk_div : out STD_LOGIC);
    end component;

    component bin2bcd
        generic (n: positive :=16);
        port ( bin_in : in STD_LOGIC_VECTOR (n-1 downto 0);
              clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              bcd0, bcd1, bcd2, bcd3, bcd4 : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    component seg7_coder
        Port ( char0, char1, char2, char3 : in STD_LOGIC_VECTOR (3 downto 0);
              clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              clk_div : in STD_LOGIC;
              cat : out STD_LOGIC_VECTOR (7 downto 0);
              an : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal clk_div: std_logic;
    signal bcd0, bcd1, bcd2, bcd3: std_logic_vector (3 downto 0);
```

Finalment, al cos de l'arquitectura, farem la instanciació dels components, introduint els paràmetres genèrics i la connexió dels ports de cadascun:

```
begin

u1: clk_divider
    generic map (eoc => 100000)
    port map (clk => clk,
              reset => reset,
              clk_div => clk_div);

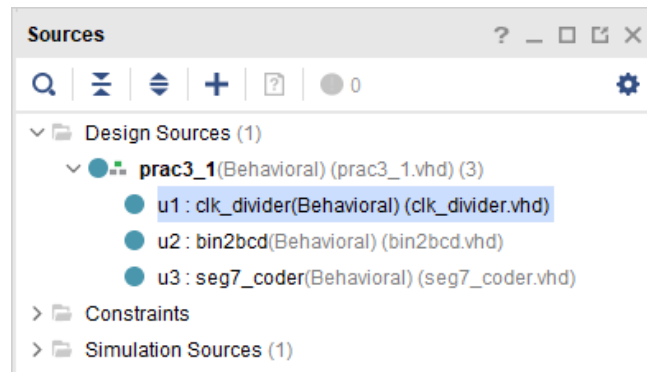
u2: bin2bcd
    generic map (n => 13)
    port map (bin_in => sw,
              clk => clk,
              reset => reset,
              bcd0 => bcd0,
              bcd1 => bcd1,
              bcd2 => bcd2,
              bcd3 => bcd3,
              bcd4 => open);

u3: seg7_coder
    port map (char0 => bcd0,
              char1 => bcd1,
              char2 => bcd2,
              char3 => bcd3,
              clk => clk,
              reset => reset,
              clk_div => clk_div,
              cat => cat,
              an => an);

end Behavioral;
```

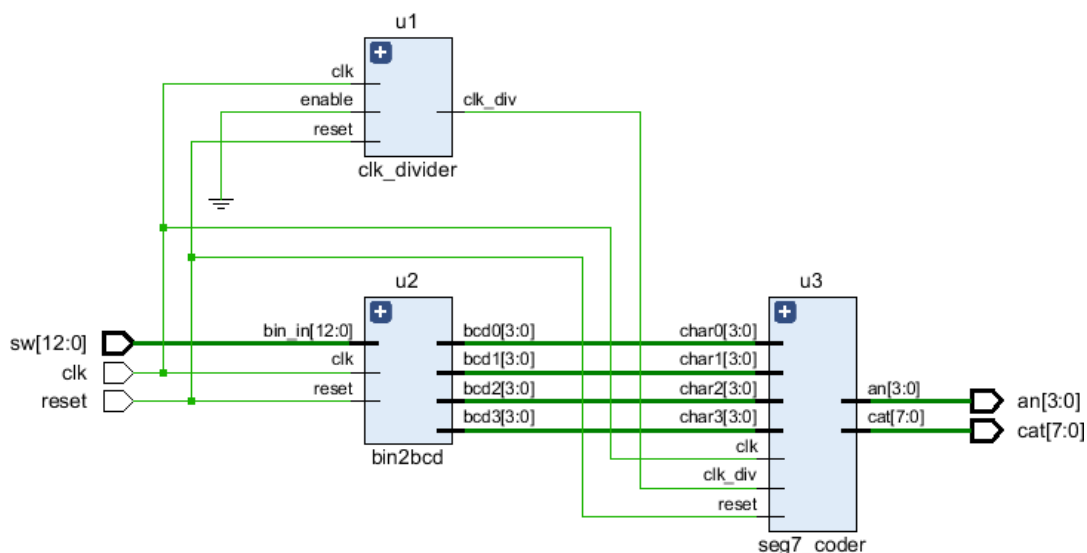
Quan incloquem, mitjançant ***add sources***, els 3 arxius vhdI corresponents a les 3 entitats a instanciar com a components, és recomanable activar la casella per fer una còpia d'aquests arxius a la carpeta del projecte, ja que així el projecte serà portable per sí sol, i davant qualsevol modificació, deixarem intactes els codis font dels projectes originals.

L'arbre jeràrquic del projecte acabarà tenint el següent aspecte:



Ara haurem de seguir tots els passos fins a generar el bitfile per a poder configurar la FPGA: Elaborated Design, Synthesis, Implementation, Generate Bitstream.

Quan obrim el Elaborated Design, si fem click a Schematic, podrem veure el nostre disseny estructurat:



Pel que fa a l'assignació de pins, com que serà exactament la mateixa que en la pràctica anterior, podem aprofitar el mateix arxiu .xdc, sempre i quan haguem conservat el nom dels ports. Seleccionar també aquí la opció de copiar l'arxiu dins del directori del projecte actual.

Per a fer aquest disseny estructurat, els arxius .vhd corresponents als 3 components es podrien haver creat directament des d'aquest projecte, no és necessari crear un projecte independent per a cadascun, però és interessant tenir-los creats per a la segona part de la pràctica.

Si es creen els arxius des del mateix projecte, per a simular-los independentment l'un de l'altre, es pot fer botó dret sobre el que estem treballant i activar la opció **Set as Top**, tant en l'espai de Design Sources com a Simulation Sources.

2a part: Block Design, empaquetat mitjançant IP

Un altre mètode per a fer un disseny estructural, consisteix en l'ús de IP's (Intellectual Property). Les IP's són components ja definits i descrits, rigorosament provats i validats, dels quals els fabricants de dispositius lògics programables en distribueixen àmplies llibreries.

Dins l'entorn Vivado hi ha un catàleg de IP's (IP Catalog, dins del project manager), dins del qual nosaltres podem triar el component que ens sigui útil, configurar-lo per adaptar-lo als requeriments del nostre disseny, i finalment incloure'l dins del nostre projecte, ja sigui mitjançant una instanciació tradicional dins d'un codi VHDL, o bé fent ús del disseny amb blocs, connectant-lo com a un símbol (caixa negra) dins d'un entorn esquemàtic.

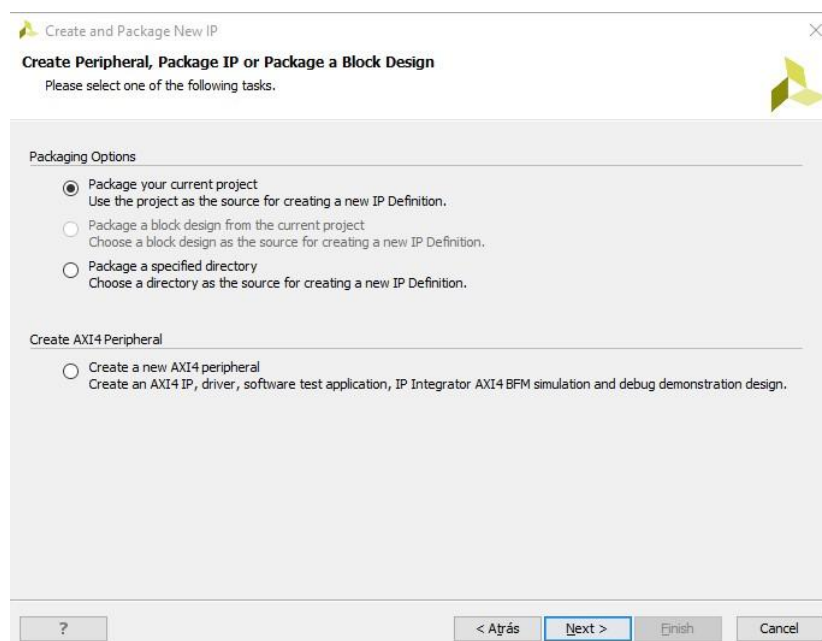
L'ús de les IP's com a unitats de disseny, pot accelerar moltíssim el procés de desenvolupament d'un projecte, ja que dins dels catàlegs podem trobar una gran varietat de components ordenats i classificats segons el seu àmbit d'aplicació. Un dels exemples més representatius de IP disponibles en el catàleg de Xilinx, és el ja clàssic MicroBlaze, un processador escalable de 32 bits que podem incorporar en el nostre disseny, connectar-lo als perifèrics necessaris, i exportar el conjunt del hardware cap al Xilinx Software Development KIT (SDK), des d'on podrem desenvolupar aplicacions de software utilitzant C i C++. *Evidentment, no és estrictament necessari utilitzar aquest programa per a fer això...

A nivell d'aprenentatge de llenguatges de descripció de hardware, l'inconvenient (si se'm permet considerar-ho com a tal) de l'ús de IP's predefinides, és que no podem accedir a la descripció hdl que el descriu, i per tant, només els podem veure com a caixa negra i conèixer-ne "tan sols" la seva funcionalitat.

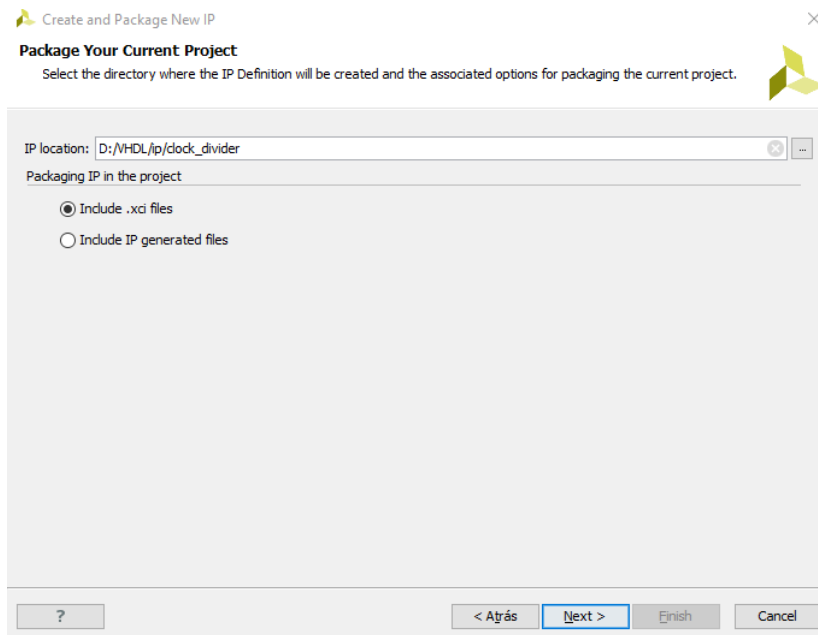
A part de les IP's ja definides, nosaltres podem crear un bloc IP a partir de qualsevol entitat que vulguem reutilitzar i inserir-la en un disseny de blocs. Això és el que farem amb els 3 components que hem aïllat anteriorment, i és per això que hem creat un projecte independent per a cadascun d'ells.

Aquesta segona part de la pràctica, consistirà en crear un repositori on guardarem totes les IP's pròpies, que generarem a partir dels tres projectes creats en la primera part. Llavors crearem un nou projecte (**top level**), i sense necessitat d'escriure ni una sola línia de codi, inserirem els tres components empaquetats com a IP, configurarem els seus paràmetres genèrics, i els interconnectarem entre ells i cap als ports de l'entitat **top level** mitjançant un entorn esquemàtic. Ja només ens caldrà simular, assignar constraints, fer la síntesi, implementació, generació de bitstream i finalment configuració de la FPGA per a comprovar que el comportament final sigui exactament el mateix que fet amb els mètodes anteriors.

Començarem pel projecte corresponent al divisor de rellotge. Amb el projecte obert, anirem a **Tools – Create and Package New IP, Next...**

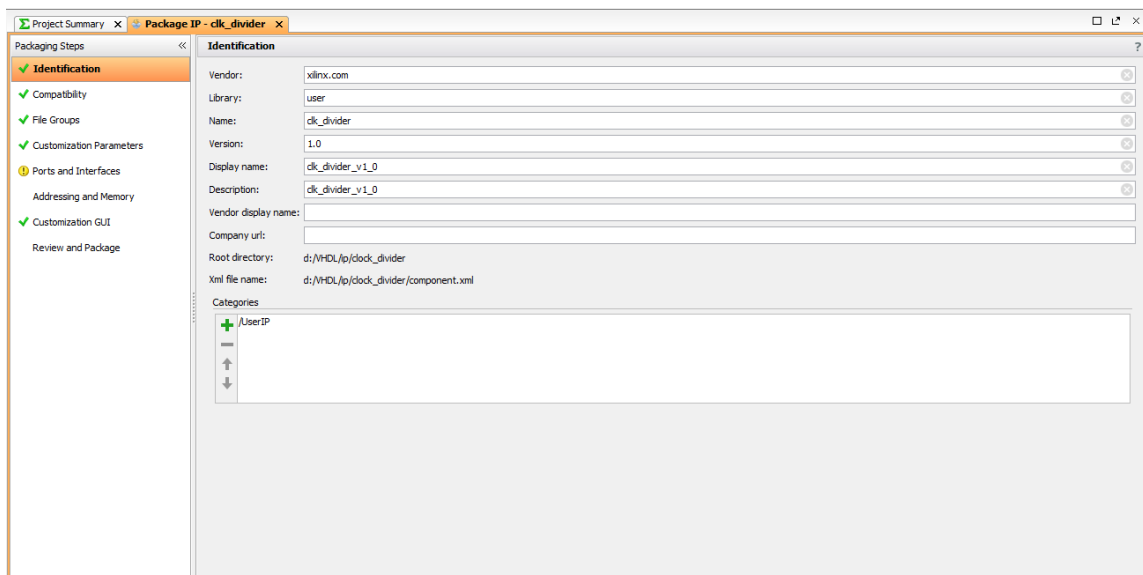


Triarem la primera opció, **Package your corrent project**.



A **IP location** triarem la ubicació on guardarem totes les nostres IP's personals. És recomanable crear una subcarpeta per a cadascuna de les IP's. A la següent finestra, OK, i Finish.

Se'ns obrirà un projecte nou de Vivado amb tots els passos per configurar la nova IP. Si repassem tots els apartats veurem que podem configurar el nom, la descripció, la versió, la compatibilitat amb les diferents famílies de dispositius, la configuració dels paràmetres genèrics, la dels ports, etc.

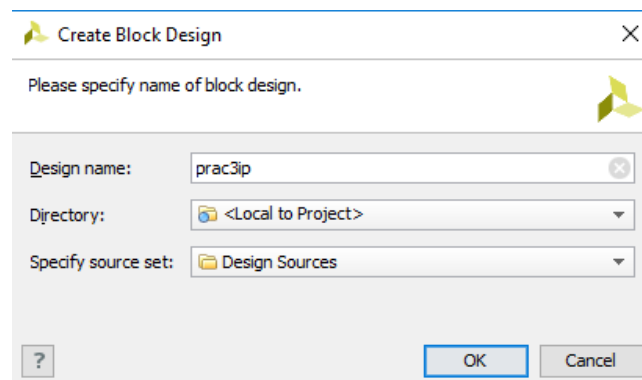


Provablement ens avisarà d'un **Warning** en l'apartat de **Ports and Interfaces**, això és degut a que no hem assignat cap clk a les constraints del projecte, però no li donarem importància.

A la finestra de l'últim apartat **Review and Package**, ens fa un petit resum de la nostra IP. A la part inferior, farem click a **Package IP**, i un cop hagi acabat ens demanarà si volem tancar el projecte, li direm que sí.

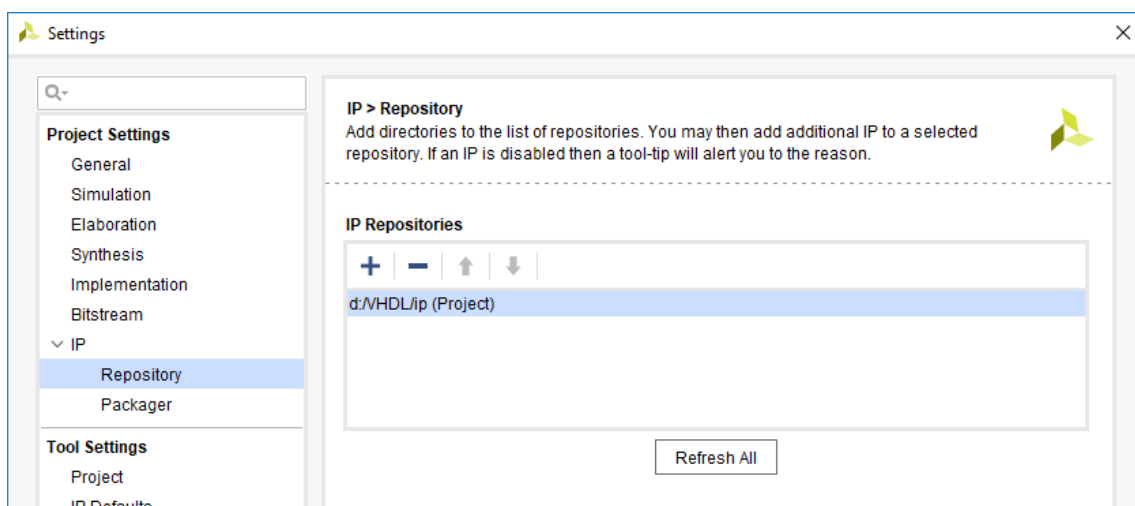
Procés d'empaquetatge finalitzat, ara farem el mateix amb els altres dos projectes.

Un cop creades les nostres tres noves IP's, crearem un projecte amb disseny de blocs per a connectar-les entre elles i crear l'entitat **top level** de la jerarquia. Per a això, crearem un nou projecte RTL sense especificar cap codi font inicialment (podem seleccionar la casella **Do not specify sources at this time** a la finestra de **Project Type**). Un cop creat i obert, a l'apartat de **IP Integrator** (a sota del **Project Manager**) farem click a **Create Block Design**.



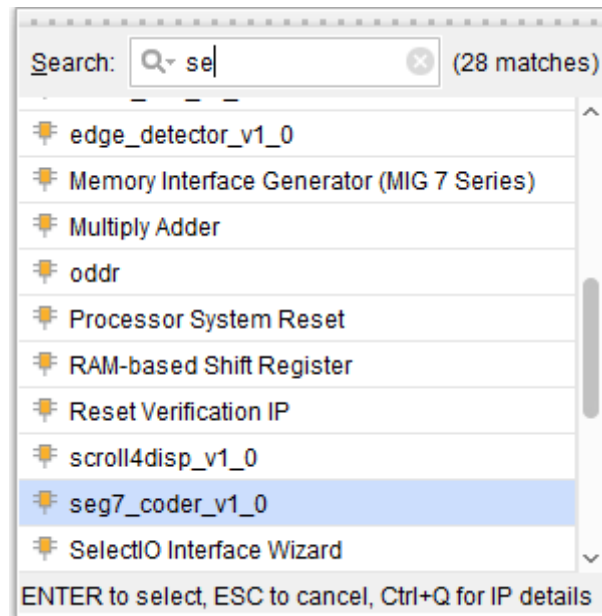
Introduïrem el nom del disseny de blocs i OK. S'obrirà una finestra on podrem introduir els elements del nostre esquema de forma gràfica.

El primer que hem de fer és anar a l'apartat **IP de Settings**, i al subapartat de **Repository**, afegir la ruta del directori on guardem les nostres IP's.

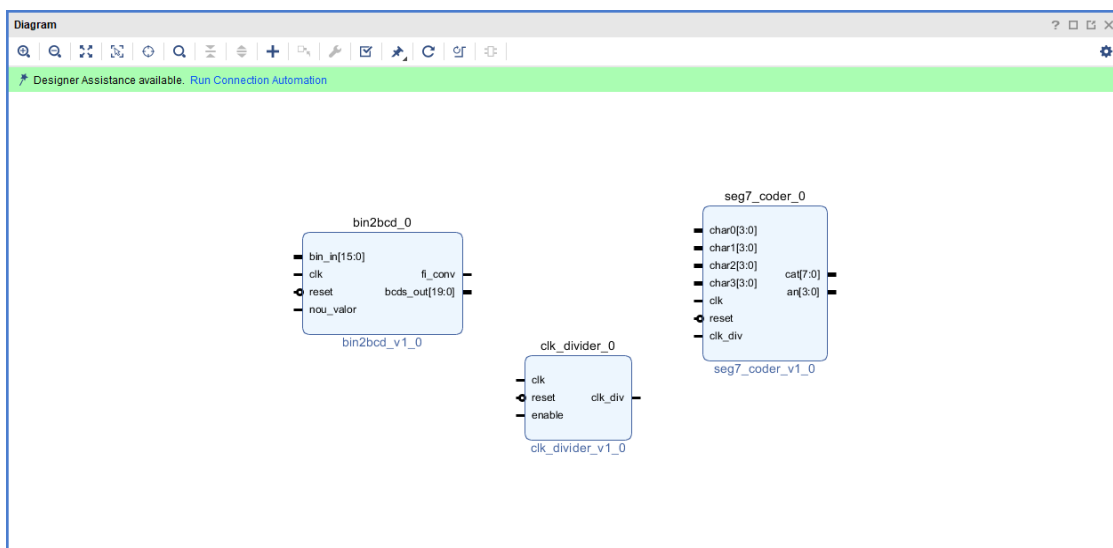


Fem Apply i OK, i ja podrem començar l'esquema.

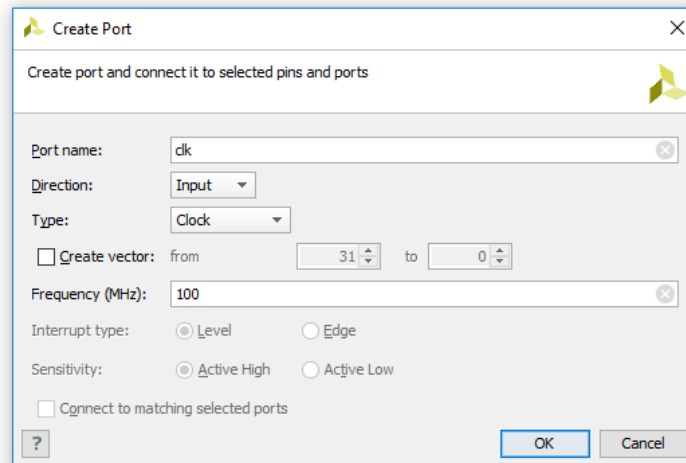
Utilitzant la icona de **Add IP** (+) afegirem les 3 IP's que hem creat anteriorment. Si hem introduït bé l'adreça del nostre repositori, haurien de sortir a la llista juntament amb les predefinides.



...i a així anar fent amb les altres.



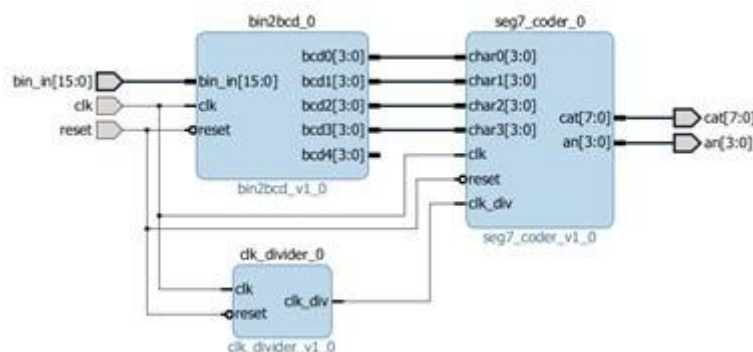
Per connectar-los entre ells és tan simple com seleccionar el port amb el botó principal del ratolí i arrossegar fins al port de l'altre component. Pel que fa als ports del disseny final, és a dir els ports que es connecten amb l'exterior (pins de la FPGA), podem fer botó dret en qualsevol lloc del canvas i triar la opció **Create Port**:



I si es tracta d'un port múltiple, seleccionarem la opció **Create Vector** i especificarem el nombre de bits.

Si el nom del port extern que volem crear coincideix amb el d'un port d'un component, podem fer botó dret sobre aquest port i seleccionar la opció **Make External** (o Ctrl+t), i ja ens crearà el port amb el nom corresponent i ens el connectarà.

Un cop fetes totes les connexions, podem fer click a la icona **Regenerate Layout** (3era icona des de baix, de la finestra del diagrama) i ens reordenarà els components i les connexions per deixar el disseny estèticament més clar. Ens quedarà més o menys així:



Si ens sembla tot correcte, farem click a la icona de **Validate Design** (5a icona començant per la dreta) i ens farà una comprovació del disseny. Després, Ctrl+S per guardar.

Un cop acabat l'esquema necessitem un arxiu que ens faci de top level de la jerarquia, un arxiu HDL que especifiqui els components utilitzats i les connexions entre ells. Això serà bàsicament una traducció a VHDL del què hem representat mitjançant el disseny de blocs. Aquest arxiu el

crearem automàticament fent botó dret sobre el nom del disseny de blocs a la finestra **Sources**, i seleccionant la opció **Create VHDL Wrapper**. Ens apareixerà una finestra on triarem la segona opció, segons la qual, aquest arxiu serà de només lectura, i el Vivado ja se n'encarregarà de fer les actualitzacions quan sigui necessari. Aquest arxiu ens quedarà en el nivell més alt de la jerarquia a l'arbre del projecte i s'anomenarà igual que el nom del disseny de blocs seguit de “**_wrapper.vhd**”.

Pel que fa a l'arxiu de constraints (.xdc), podem reutilitzar el mateix de la pràctica 2, igual com hem fet en la primera part d'aquesta pràctica. Abans ens n'hem d'assegurar que el nom dels ports és el mateix. Si divergeixen sempre serà més fàcil i ràpid canviar el nom del port de l'esquema.

Ara ja podem realitzar els processos de síntesi, implementació, generació del bitstream i configuració de la tarja. No oblideu passar per la simulació!!!!

Si voleu aprofundir sobre el tema del disseny amb IP's de Vivado, hi ha dos documents a la carpeta corresponent a Atenea.